

Guía de estudio · Pre-defensa Wilmercito

Cómo estudiar: lee en orden 1-6. Cada sección termina "donsto" (frase lista).

1. Qué conseguiste (memoriza estos 5)

- 1 **Asistente real dentro del software:** burbuja W en Django, responde sobre tickets, usuarios, áreas, turnos e informes, con fuente citada.
- 2 **Motor propio local:** llama.cpp compilado con CUDA + Qwen2.5-3B en tu RTX 2060. Cero nube, cero Ollama.
- 3 **No inventa:** RAG sobre 2886 documentos + guardrails que frenan jailbreaks y fuera-de-tema (medido).
- 4 **Hace cosas con permiso:** crear/anotar/reclasificar/asignar/eliminar con chip de confirmación, a tu nombre + colaborador Wilmercito.
- 5 **Aprende y se controla:** votos · / · jefe cura · base; panel Asistente con métricas; backups, systemd, tests, merge a producción.

Di esto: "Integré un LLM local al sistema de soporte como asistente con lectura total, escritura con confirmación y aprendizaje supervisado, todo medido."

2. Cómo funciona (el dibujo que debes saber hacer de memoria)

```
Burbuja (Django :8011) · FastAPI /ia/* (:5012) · Chroma (VM, CPU) + reglas
· llama-server :8081 (host GPU) · respuesta con fuente
```

- **Embeddings:** convierten texto en vectores; lo parecido queda cerca. Multilingüe, normalizado = distancia coseno.
- **Umbral 0.5:** si lo mejor supera 0.5, "no tengo ese dato" (calibrado con tus datos, no copiado).
- **Intents:** saludos/identidad/estadísticas van por código exacto (rápido, 100% fiable); lo demás, a retrieval + modelo.
- **Guardrails en 3 capas:** prefiltro jailbreak · system+contexto · postfiltro. Porque el modelo chico solo no obedece.

Di esto: "El modelo solo redacta; las decisiones (qué responder, qué permitir, qué guardar) viven en código determinista y testeado."

3. Obstáculos que atravesaste (tu mejor material de defensa)

| # | Problema | Causa real | Fix | ---|-----|-----|----| | 1 | Dump PG16 ilegible en PG15 | formato custom 1.15 | SQL plano | | 2 | RAG mudo (distancias 1-12) | vectores sin normalizar | normalize_embeddings + umbral propio 0.5 | | 3 | El 1.5B contaba chistes ante jailbreak | modelo chico obedece última instrucción | guardrails en backend | | 4 | Reranker "mejor" ordenaba peor | medido: KB exacta 6ta/12 | OFF por defecto, documentado | | 5 | Fine-tuning 5/30 tres noches | 2200 tickets ahogan 15 ejemplos | gate frenó despliegue; plan Noche 4 | | 6 | Review nativa imposible | candidato 287 archivos vs presupuesto | evidencia sustituta (tests/eval/bitácora) |

Di esto: "Cada fracaso está medido y documentado; los gates existen para frenar, y frenaron."

4. Números que debes saber de memoria

- GPU: RTX 2060 6 GB; modelo 2168 MiB; ~140 tok/s
- Docs: 2886 (319 prod + 2494 histórico + KB); umbral 0.5; sugerencia 0.9
- Tests: 13 unitarios; /ia/evaluar 10/10; FT eval 5/30 (frenado)

- Train: 279·293 steps, loss 3.25·0.77, ~10 min/epoch
- Backups diarios 02:00 + restore probado (2771)

5. Preguntas probables del docente (+ respuesta corta)

- *¿Por qué no Ollama?* Motor directo = control total + puntos; Ollama es wrapper del mismo llama.cpp.
- *¿El fine-tuning falló?* Sí, 3 veces, documentado. El gate funcionó: nada roto llegó a prod.
- *¿Y si alucina?* Umbral + fuentes citadas + sugerencias marcadas como "no verificado".
- *¿Qué es tuyo vs librerías?* Arquitectura, guardrails, intents, loop, poderes, specs, mediciones. Modelos y frameworks, de la comunidad (citados).
- *¿Riesgos?* Escritura siempre con confirmación; roles respetados; auditoría (quién/cuándo).

6. Glosario mínimo

- **RAG:** responder con documentos recuperados, no de memoria del modelo.
- **Embedding:** vector que representa significado; cerca = parecido.
- **GGUF/Q4:** modelo comprimido para correr en poca VRAM.
- **QLoRA:** entrenar solo un adaptador chico (barato) sobre modelo congelado.
- **MCP:** protocolo para exponer herramientas al asistente (18 tools).
- **Chroma:** base de vectores persistente.
- **Guardrail:** regla de código que limita al modelo.
- **Threshold/umbral:** distancia máxima para aceptar un resultado.